

AM50 Topic 3: Random Walks (due Tues., March 8 at 5 PM)

Today's computer lab is as follows:

1. We will carry out simulations of random walks, and verify the $L(t) = \sqrt{Dt}$ relationship outlined in class.
2. We will read in data for the Standard and Poor's 500 Index into Python, and start to ask whether there is evidence for or against the hypothesis that the data are well described by a random walk.

1 Simulating Random Walks

1.1 Standard Random Walkers

We would like to simulate a random walk. To do this we recall the model from class. We consider a walker at $x = 0$ and flip a coin to let the walker move either to the left or to the right. This is accomplished with the following code from lecture (`random_walk.py`):

```
#!/usr/bin/python
from random import random

# Starting position
a=0

# Number of steps
steps=20

# Carry out steps
for i in range(steps):

    # Print current position
    print i,a

    # Carry out a random step
    if random()<0.5:
        a+=1
    else:
        a-=1

# Print final position
print steps,a
```

Note that the imported command `random()` chooses a random number in the half-open interval $[0,1)$. There are various ways in Python to get random numbers and whether the right endpoint of the interval is included varies.

Now we would like to run similar code for many different walkers starting at 0 and compile statistics over time. The file `random_walkers.py` does this.

```

#!/usr/bin/python
from random import random
import numpy as np
import matplotlib.pyplot as plt

# Time steps
tmax = 200

# Max spatial steps
nmax = 2*tmax

# Number of walkers and initial positions
nwalkers = 100
x = [0]*nwalkers

# Bin random walk positions into an empty 2x2 array (time, bin)
bins = np.zeros((tmax,nmax+1)) # tmax rows x nmax+1 columns
bins[0,nmax/2+1] = nwalkers # all walkers initially at x=0

# Intialize statistics
meanx = [0]*tmax
widthx = [0]*tmax

# Time iteration
for i in xrange(1,tmax):

    # Walker iteration
    for j in xrange(nwalkers):

        if random()>0.5:
            x[j] += 1
        else:
            x[j] -= 1

        # Increment count for location of x[j]
        bins[i,x[j]+int(nmax/2)-1] += 1 # recenter bins around x=0

    # Statistics
    meanx[i] = sum(x)/float(len(x))
    widthx[i] = np.std(x)/np.sqrt(2)

# Sample plots -- very crude!
def walker_hist(i):
    plt.figure(i)
    binx = np.arange(np.shape(bins)[1]-np.shape(bins)[1]/2-1.5
    biny = bins[i,:])
    plt.bar(binx,biny,width=1) #width here is just a plot style option
    plt.xlim([-nmax/40, nmax/40])

walker_hist(0),walker_hist(1),walker_hist(100)

```

Please note the following aspects of this program:

1. The program simulates the trajectories of `nwalkers` random walkers. This is so that we can do the random walk many times.
2. The program contains two *loops*. Within the loop the program repeats the specified task a specified number of times. The outer loop goes through the different time steps. The inner loops moves each walker separately. Note we have used `xrange` in place of `range` in the loop which is often faster for long loops and a bit more memory efficient.
3. The plotting commands are just an example. You should add annotations (e.g. labels, title, etc.).

Exercise 1

Run this computer program (saved as `random_walkers.py` in your home directory). The variable `widthx` saves the (half) width of the distribution as a function of time—this is calculated from the standard deviation of the distribution in the positions of the walkers at each time. Plot the width `widthx` and convince yourself that the width grows according to our famous law

$$L(t) = \sqrt{Dt}.$$

A good way to convince yourself of the $\sqrt{Dt}$ law is to use a log–log plot. Note that if we calculate

$$\log(L(t)) = \log(\sqrt{Dt}) = \frac{1}{2} \log(Dt) = \frac{1}{2} \log(t) + \frac{1}{2} \log(D).$$

Thus if we plot $\log(L)$ versus $\log(t)$ we should find a straight line with a slope of $\frac{1}{2}$.

We also might check that the value of the diffusion constant is what we expect: In class we argued that $D = \frac{a^2}{2\tau}$, where a is the lattice spacing and τ is the timestep. In our simulation both the lattice spacing $a = 1$ and $\tau = 1$ so that $D = \frac{1}{2}$.

Exercise 2

(A) Do a log–log plot of $L(t)$ and verify that the slope is $1/2$ in accordance with the prediction. Start with the command `plt.loglog(widthx)`. (B) Plot the average value of the walkers over time. What happens to the mean? Does this make sense? (C) Let's check that the diffusion constant is correct. First plot the width and then plot the prediction.

1.2 Biased Random Walkers

What happens if we change the probability of moving to the left or the right? Modify your code to use the test `if random()>R` in place of `if random()>0.5`.

Exercise 3

What happens if $R = 1$? $R = 0$? $R = 0.75$? Can you explain what is going on with these behaviors? HINT: Plot the mean as a function of time in these different cases and see what happens. Also see if there is a change in the behavior of the standard deviation. Can you explain what is going on?

Exercise 4

Can you think of an example where a biased random walker might be an appropriate model? Discuss with your partner and your TF and brainstorm what might work.

2 Fun with Stock Data

Can we apply these idea to data from Stocks? To read in data from the S&P 500 for 50 years, adapt your csv file reading and splitting code from the Google Flu lab to read in the file `S_P_500.csv`. (Make sure you don't accidentally resave this file in Microsoft Excel. You can open it in a simple text editor if you wish. If you run into problems just redownload the file.)

This file contains data from January 3, 1953 to February 19, 2010 in *reverse* chronological order. The data are saved for every day that the Stock Market is open. The *0th* column stores the dates. The first column gives the opening price; the second column gives the high; the third column gives the low and the fourth gives the close. The fifth column gives the volume of shares traded.

For some strange reason the data are stored backwards—the early dates are at the end of the table and the end dates are at the beginning. So you must be careful when plotting and analyzing the data.

Now, plot the data in the first column (opening price). The data look to be quite exponential! This makes sense of course, you are usually told to invest in stocks (or even a bank) because of exponential growth. And even inflation is exponential. A simple first bit of analysis to do is to see how well the data are fit by the law

$$S(t) = S_0 e^{rt},$$

where $S(t)$ is the price of the stock portfolio at time t , S_0 is the initial price and r is the growth rate.

Recall from earlier, the way to test that something is an exponential we should take the logarithm of the equation: We have

$$\log(S(t)) = \log(S_0 e^{rt}) = \log(S_0) + rt.$$

Then, if we plot $\log(S)$ versus t , the data should fall on a straight line.

Exercise 5

To test this theory, let's make a semi-log plot of the stock data using the command `plt.semilogy`. You will see that the data are basically along a straight line. Demonstrate that the data are pretty well fit by the law $S(t) = 20e^{rt}$, with $r = 3 \times 10^{-4}$. According to this law, how much should Harvard's ~ 20 billion dollar endowment increase in size during a decade?

Exercise 6

Discuss how you might decide if the data are consistent with the random walk model.